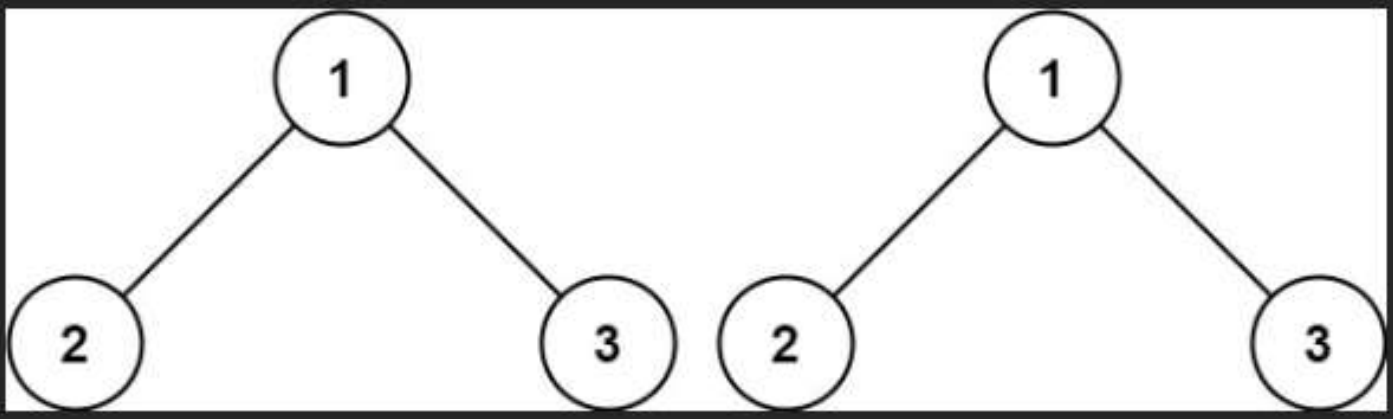
100. Same Tree

Easy | Topics | Companies

Given the roots of two binary trees `p` and `q`, write a function to check if they are the same or not.

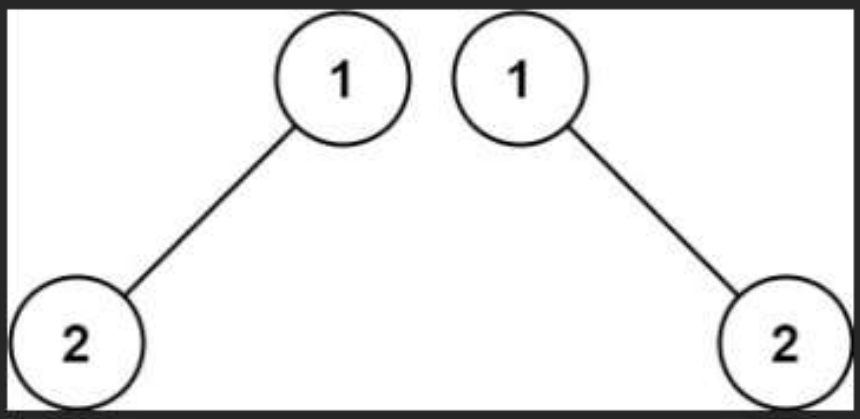
Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1:



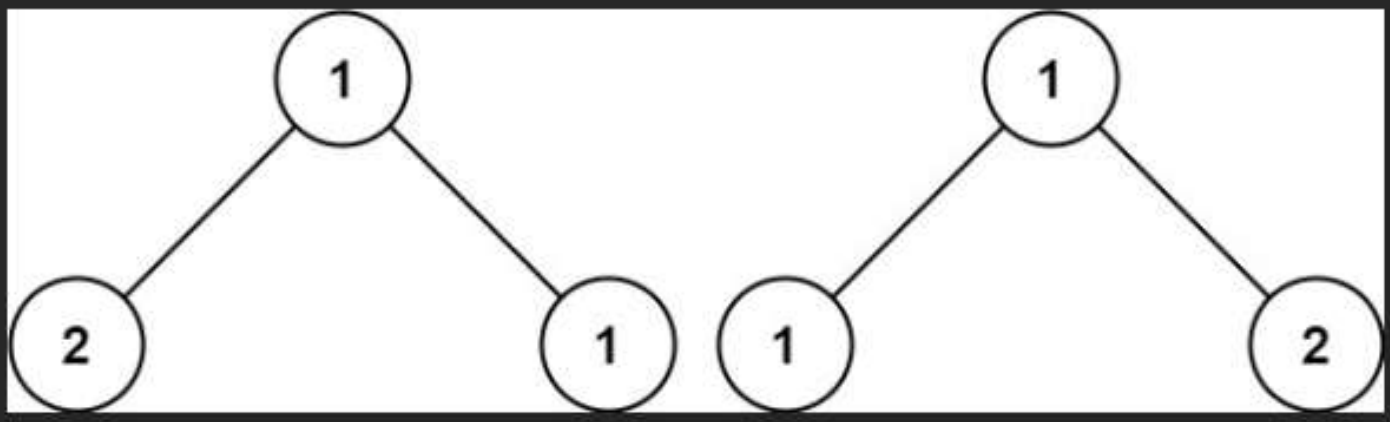

```
Input: p = [1,2,3], q = [1,2,3]
Output: true
```

Example 2:



> **Input:** p = [1,2], q = [1,null,2]
> **Output:** false

Example 3:



> **Input:** p = [1,2,1], q = [1,1,2]
> **Output:** false

Constraints:

- The number of nodes in both trees is in the range `[0, 100]`.
- `-10^4 <= Node.val <= 10^4`

RS / Day-98 / 14-11-26.

Leetcode - 100. Same Tree

Given the roots of two binary trees, p & q, write a function to check whether they are the same or not.

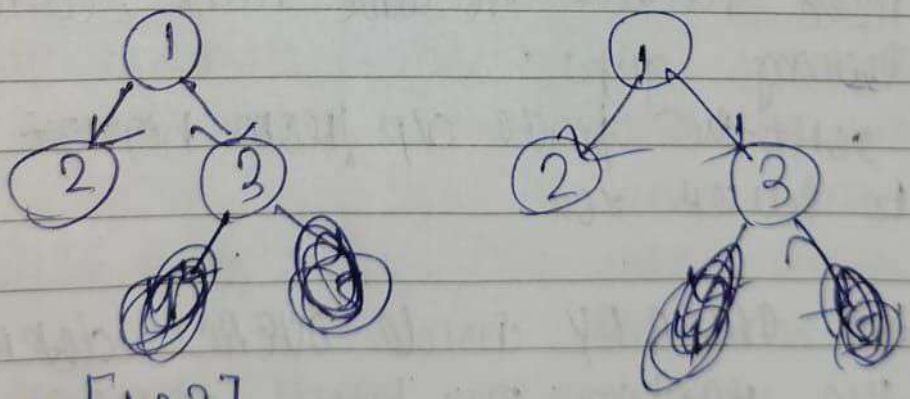
Two binary trees are considered the same if: 1) they have the exact same structure

2) the corresponding nodes have the same values.

Return true if the trees are identical; otherwise return false.

Even if both trees contain the same values, they are not necessarily the same tree if their structures differ.

Ex 1



p = [1,2,3]
q = [1,2,3], o/p true.

explanation ⇒ Root nodes : 1=1
left children = 2=2
right children : 3=3
both trees have identical structures, return true.

Ex-2 input → p = [1, 2]
q = [1, null, 2]

o/p false explanation: both trees contain the values 1 & 2, but their structures are different.
→ in the 1st tree, 2 is the left child, → in the 2nd tree, 2 is the right child.
therefore, return false.

conditions ① both nodes are NULL → both positions are empty, so they match.
② only one node is null → structures differ
③ both nodes exist → compare their values & children.

Approach → Recursive DFS (depth 1st search)
we compare the two trees recursively.
for every pair of corresponding nodes:
1. if both nodes are NULL, they are identical at this position. Return true.
2. if one node is NULL, & the other is not, their structures differ, return false.
3. if their values are different, return false.
4. otherwise, recursively compare:
• left subtree of p with left subtree of q.
• right " " , right " "

class Solution { public: → represents the root of the 1st tree
bool isSameTree(TreeNode* p, TreeNode* q) {

if (p == NULL && q == NULL) return true;

if both r empty, their structures match at this position.

checks whether only one of the nodes is NULL. if (p == NULL || q == NULL) return false;

if one tree has a node & the other doesn't, their structures are different.

checks whether the values of the current nodes are diff. if they are diff, the trees cannot be identical. if (p->val != q->val) return false;

we recursively compare:- • left child of the 1st tree with left child of the 2nd tree. • right child of the 1st tree with right child of the 2nd tree.

return isSameTree(p->left, q->left) && isSameTree(p->right, q->right);

calling the function again & again

}

};

if either subtree differs, the complete result becomes false.

T.C = O(N) | O(min(N,M)) → N & M are the numbers of nodes in the 2 trees.

S.C = O(H)

balanced tree = O(log N)

skewed tree = O(N)

space is used by the recursive call stack.

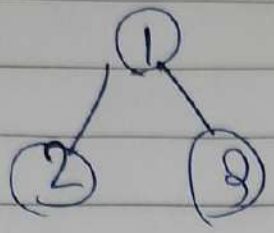
in worst case, we visit every node in both trees.

n is the no. of nodes in the smaller tree when all corresponding nodes match.

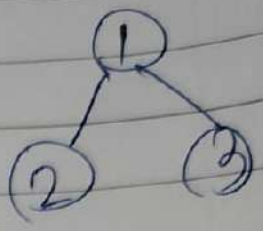
Any Two || input:- p = [1,2,3]
q = [1,2,3]

Tree representation:-

Tree p



Tree q

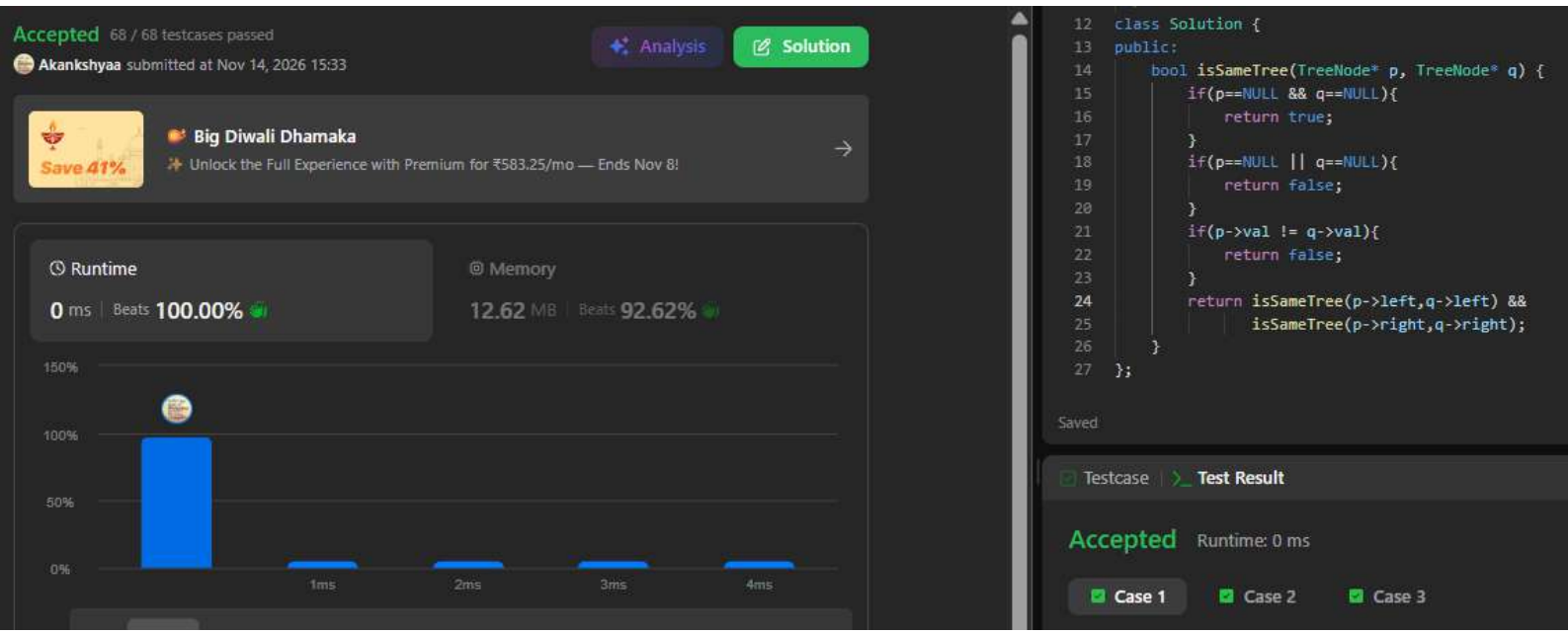


all corresponding nodes match.

step	comparison	result
1	1==1	match
2	2==2	"
3	left children of 2 are NULL	"
4	right " " " " "	"
5	3==3	"
6	left children of 3 are NULL	"
7	right " " " " "	"

The recursion reaches the end of both trees,

final o/p → true.




```cpp
class Solution {
public:
    bool isSameTree(TreeNode* p, TreeNode* q) {
        if(p==NULL && q==NULL){
            return true;
        }
        if(p==NULL || q==NULL){
            return false;
        }
        if(p->val != q->val){
            return false;
        }
        return isSameTree(p->left,q->left) &&
               isSameTree(p->right,q->right);
    }
};
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3